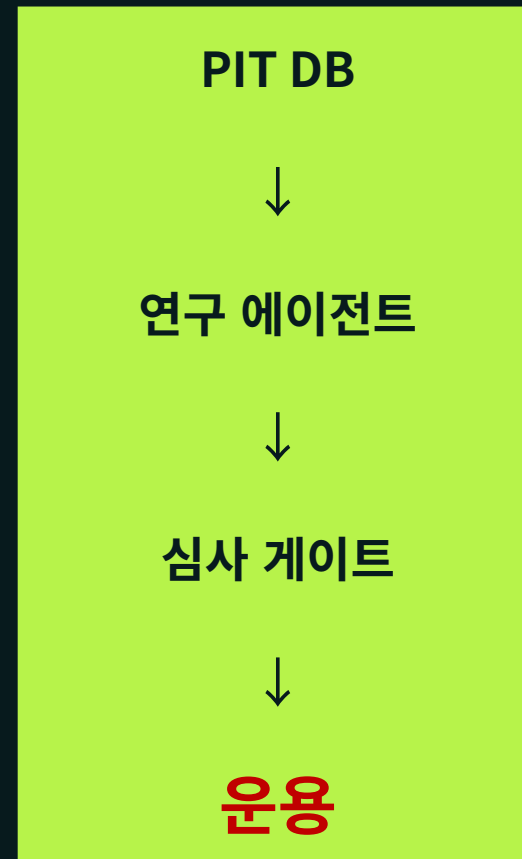


Claude Code로 구현하는 AI 헤지펀드

에이전트는 연구하고, 코드는 심사하고, 사람은 승인합니다



네 단계로 구현을 설명합니다

01 —

설계 원칙

LLM을 어디에 두나

5계층 · 권한 분리 · 사람 승인

02 —

저장소와 에이전트

누가 일하나

저장소 구조 · 13개 역할 · 권한

03 —

연구와 심사

무엇을 믿나

사전등록 · DSR 게이트 · 리스크 중립화

04 —

매크로 운영

매일 어떻게 도나

cron · 토론 · 결정 로그 · 로드맵

LLM의 판단은 코드가 정한 게이트를 통과할 때만 돈이 됩니다

01

설계 원칙

에이전트의 권한을 계층별로 나눈다

LLM은 연구 계층에만 두고 승인은 코드와 사람이 말합니다

데이터 → 연구 → 심사 → 실행 → 리스크



LLM은 연구 계층에만 두고, 승인과 주문은 코드와 사람이 맡는다

※ 원문 조직(CIO·CRO·CTO)을 이 다섯 계층에 대응시킨다

세 가지 금지 규칙이 시스템의 경계를 정합니다

CLAUDE.md에 적고 hooks로 강제한다

RULE 1

홀드아웃 금지

- 등록 전 접근 차단
- 기간은 카드에 고정

RULE 2

결과 수정 금지

- 심사자는 읽기 전용
- 재실행만 허용

RULE 3

주문 금지

- 주문 API 차단
- 사람이 최종 승인

에이전트가 할 수 없는 일을 먼저 정해야 자율화가 안전합니다

02

저장소와 에이전트

Claude Code의 프로젝트 구조로 조직을 옮긴다

저장소 하나가 펀드의 조직과 규칙을 담습니다

에이전트·스킬·hooks는 .claude/ 아래, 계산 엔진은 core/ 아래에 둔다

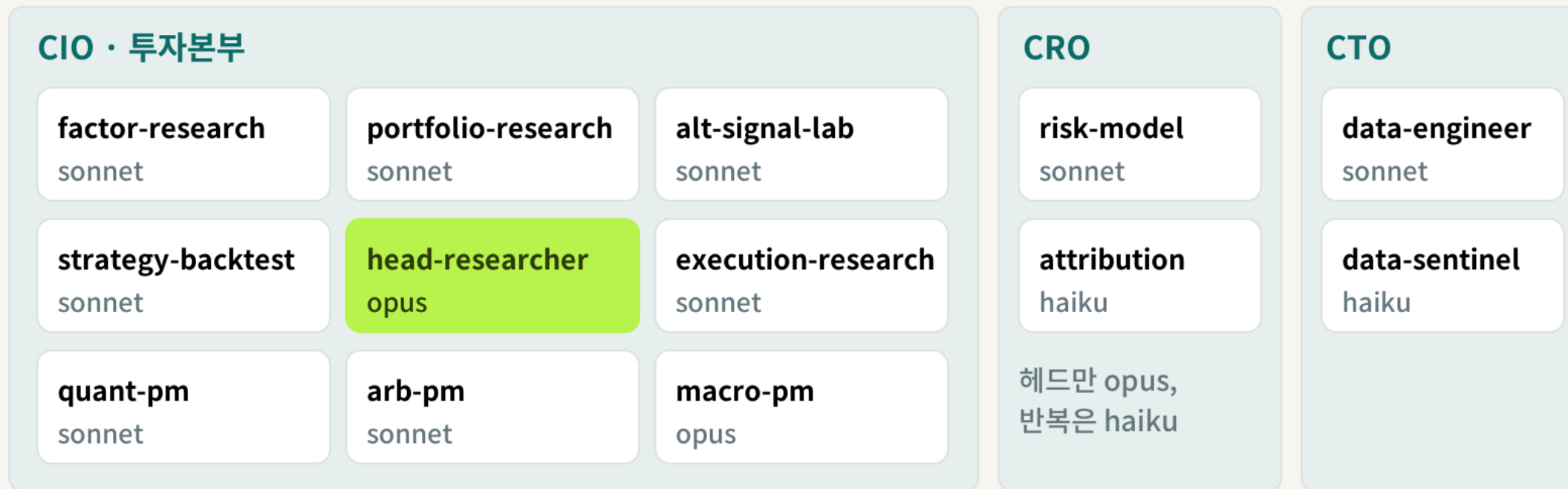
저장소 구조

```
quantfund/  
├─ CLAUDE.md           # 운용 헌장 · 금지 사항  
├─ .claude/agents/     # 13개 역할 에이전트  
├─ .claude/skills/     # /propose /backtest /review /morning  
├─ .claude/settings.json # hooks · 권한  
├─ core/ pit · backtest · stats · registry · risk · costs  
├─ research/registry/  # 사전등록 카드 (YAML)  
└─ logs/decisions/     # 결정 로그 (JSONL)
```

※ core/ 모듈은 결정론적 파이썬 — LLM이 결과 숫자를 만들지 않는다

13개 역할 에이전트에 작업 난이도별로 모델을 배정합니다

.claude/agents/*.md frontmatter의 model 필드



※ 모델 배정은 제안 — 성적표로 비교해 조정한다

심사 에이전트에게는 읽기 권한만 줍니다

tools 필드에서 Edit·Bash를 빼면 결과를 고칠 수 없다

```
.claude/agents/head-researcher.md
```

```
---
```

```
name: head-researcher
```

```
description: 등록 카드와 백테스트 결과로 채택·기각을 판정
```

```
tools: Read, Grep, Glob          # Edit·Bash 없음
```

```
model: opus
```

```
---
```

- 결과를 고칠 수 없고 읽기만 한다
- DSR·표본 외·비용 반영 후 성과와 경제 논리를 함께 본다
- 판정과 근거를 logs/decisions/ 형식으로 반환한다

※ 채택·기각 근거는 결정 로그로 남겨 사후 감사가 가능하다

반복 업무는 네 개의 스킬로 표준화합니다

.claude/skills/<이름>/SKILL.md — 슬래시 명령으로 호출한다

/propose

가설 등록

등록 카드 생성

/backtest

검증 실행

PIT·비용 반영

/review

헤드 심사

게이트 판정

/morning

모닝미팅

매크로 운영

같은 절차를 같은 방식으로 돌려야 성과를 비교할 수 있습니다

03

연구와 심사

사전등록과 게이트로 다중검정을 통제한다

연구 사이클은 등록에서 시작해 게이트에서 걸러집니다

기각된 시도도 시험 횟수 N에 그대로 포함된다



백테스트 전에 가설과 시험 범위를 등록합니다

등록 시각과 해시가 사후 수정을 막는다

```
research/registry/KR-VAL-2026-014.yaml
```

```
id: KR-VAL-2026-014
```

```
hypothesis: 정정 반영 E/P가 최초값 E/P보다 IC가 높다
```

```
data: fin_pit as-of, 2002-01 ~ 2019-12    # 2020~ 홀드아웃
```

```
grid: {lag: [0, 1], neutral: [sector, size]}    # 시험 4개
```

```
metric: 롱숏 샤프, 비용 반영
```

```
registered_at: 2026-10-04T09:12+09:00
```

```
hash: 3f9a...c21
```

※ 예시 카드 — 2020년 이후는 홀드아웃으로 봉인한다

게이트는 다중검정과 비용을 보정한 뒤 판정합니다

core/stats.py가 등록부의 N으로 DSR을 계산한다 · 기준값은 제안

DSR

$$DSR = \Phi \left(\frac{(\widehat{SR} - SR_0) \sqrt{T-1}}{\sqrt{1 - \hat{\gamma}_3 \widehat{SR} + \frac{\hat{\gamma}_4 - 1}{4} \widehat{SR}^2}} \right)$$

통과 조건 (제안)

DSR

0.95 이상

표본 외 샤프

표본 내의 50% 이상

비용 반영 후

샤프 0 초과

게이트를 통과하지 못한 전략은 헤드 심사에 올라가지 않습니다

운용팀 포트폴리오는 리스크 모델로 중립화합니다

계약: 섹터·스타일 노출 0 · 순노출 0 · 총 레버리지 한도

CRO · risk-model 에이전트가 계약을 공급

$$\max_w \alpha^\top w - \frac{\lambda}{2} w^\top \Sigma w \quad \text{s.t.} \quad B^\top w = 0, \quad \mathbf{1}^\top w = 0, \quad \|w\|_1 \leq L$$

SIGNAL

채택 전략의 신호

RISK

섹터·스타일 노출

OUTPUT

중립 포트폴리오

최적화는 코드가 풀고, 에이전트는 입력과 해석을 말합니다

hooks가 홀드아웃·주문·로그 규칙을 강제합니다

PreToolUse에서 exit 2로 차단하고, Stop에서 결정을 기록한다

hooks — 가드레일

PreToolUse — 등록 전 홀드아웃 접근 차단

```
[[ "$p" == *holdout/* ]] && ! registered "$id" && exit 2
```

PreToolUse — 주문 API·push 는 에이전트가 못 한다

```
[[ "$cmd" == *broker*|"git push"* ]] && exit 2
```

Stop — 세션 결과를 결정 로그로 남긴다

```
jq -c '{ts,agent,model,input_hash,decision}' >> logs/decisions.jsonl
```

※ registered·broker 판정 함수는 core/registry.py에 둔다

04

매크로 운영

원문 01~07 파이프라인을 Claude Code로 옮긴다

매일 아침 파이프라인은 cron과 headless 실행으로 돕니다

단계마다 --model로 비용을 나눈다

매크로 파이프라인 — cron + claude -p

06:40 수집·번역 (저가 모델, LLM 없는 단계는 python)

```
40 6 * * 1-5 python pipeline/collect.py && \
    claude -p "/translate" --model haiku
```

07:00 모닝미팅 · 리스크 성적표

```
0 7 * * 1-5 claude -p "/morning" --model sonnet --output-format json
```

07:20 Bull·Bear 토론 → PM 결정 (구조화 출력)

```
20 7 * * 1-5 claude -p "/debate SPY QQQ TLT GLD" --model opus \
    --output-format json --json-schema schema/decision.json
```

※ 수집·발행처럼 판단이 없는 단계는 LLM 없이 python으로 처리한다

PM 결정은 JSON 스키마로 받아 그대로 기록합니다

--json-schema를 주면 결과가 structured_output 필드로 온다

schema/decision.json — PM 결정 형식

```
{ "type": "object",
  "properties": {
    "asset":      {"type": "string"},
    "signal":     {"enum": ["Buy", "Hold", "Sell"]},
    "conviction": {"enum": [1, 2, 3]},
    "evidence":   {"type": "array", "items": {"type": "string"}} },
  "required": ["asset", "signal", "conviction", "evidence"] }
```

※ 자유 서술 대신 정해진 형식이어야 성적표 채점이 자동화된다

LLM 판단은 학습 종료 이후의 기록으로만 평가합니다

사전학습 누수 — 모델이 과거 결과를 기억할 수 있다

BACKTEST

- 학습 기간 포함
- 결과 기억 가능
- 참고용

성과 증거로 쓰지 않는다

FORWARD

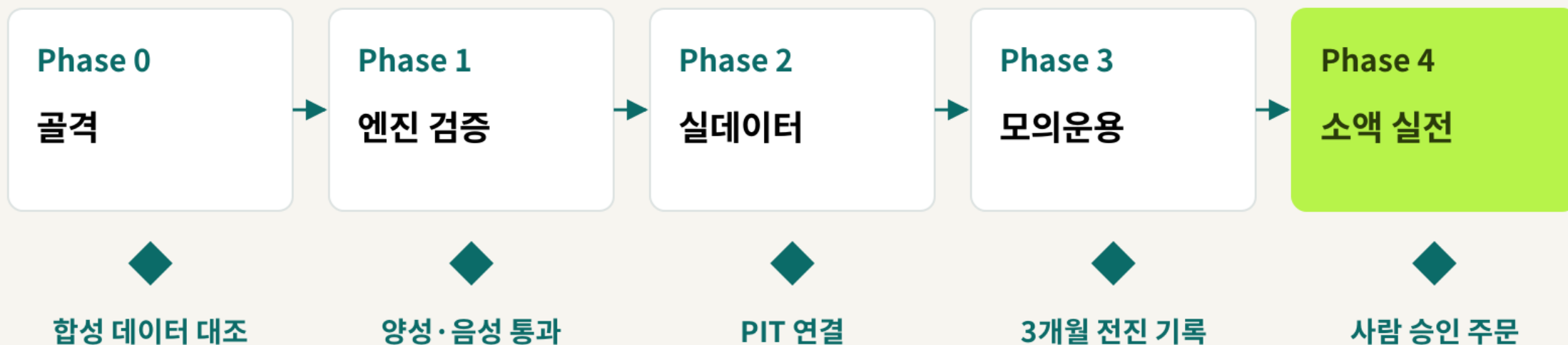
- 결정 로그 누적
- 입력 해시 보존
- 모델 버전 기록

이 기록만 성과로 인정

모델을 바꾸면 성적표도 새로 시작합니다

구현은 다섯 단계로 나누고 게이트로 넘깁니다

엔진 검증 → 실데이터 → 모의운용 → 소액 실전



◆ 게이트 — 통과해야 다음 단계로 넘어간다 (기준은 제안)

AGENTS RESEARCH.

CODE DECIDES THE GATE.

HUMANS APPROVE ORDERS.

자율화의 범위는 hook과 게이트가
정하고, 기록이 그것을 증명합니다.